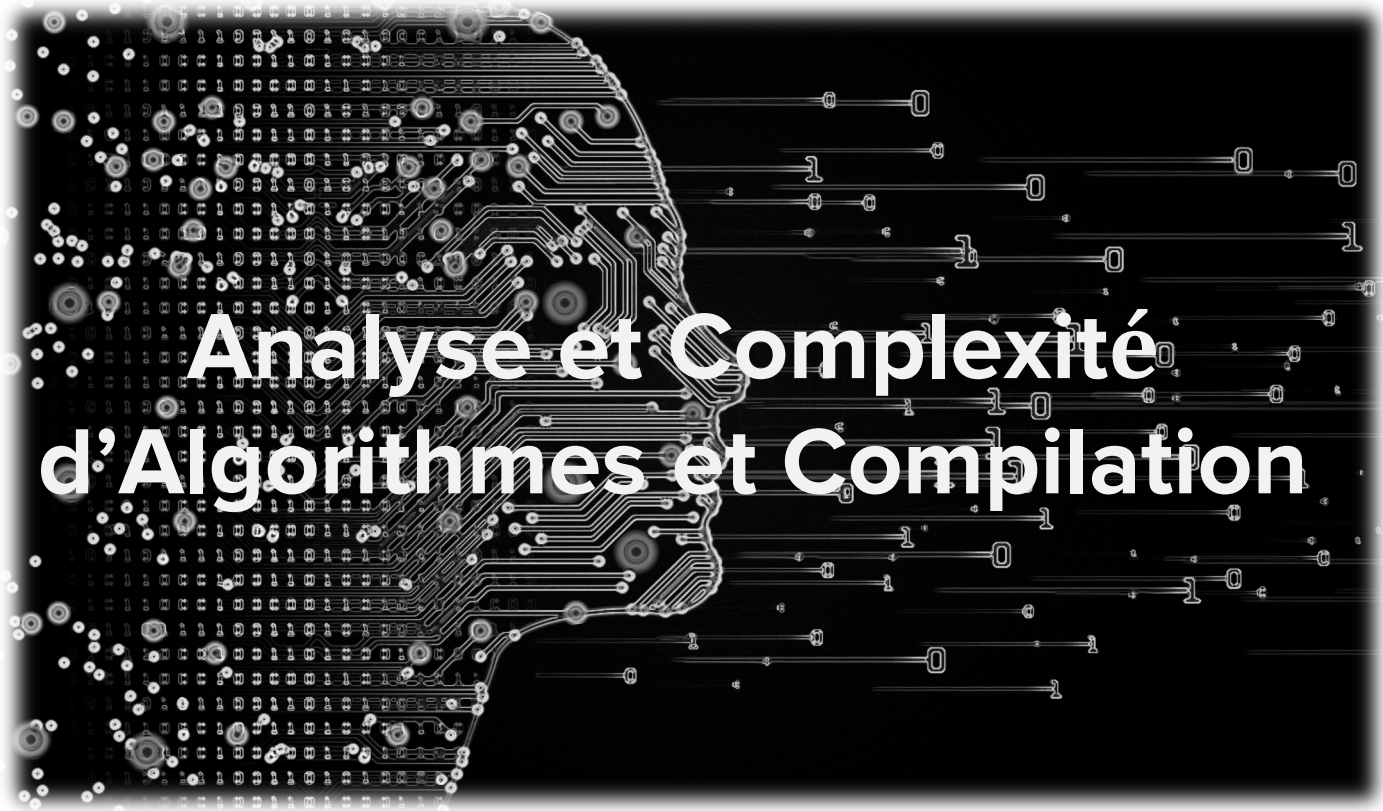




Analyse et Complexité d'Algorithmes et Compilation

Chapitre 3 : Algorithmes de Tri partie 1

Plan

- ❑ Introduction
- ❑ Applications
- ❑ Différents types de tri
- ❑ Tri à bulle
- ❑ Tri par sélection
- ❑ Tri par insertion
- ❑ Exercices
- ❑ Tri par fusion
- ❑ Tri rapide
- ❑ Optimalité des algorithmes de tri
- ❑ Exercices

Introduction: Problème de Tri

- ❑ Un des problèmes algorithmiques les plus fondamentaux.
- ❑ Un algorithme de tri est un algorithme qui prend en entrée un tableau ou une liste de nombres, et qui trie les nombres du tableau ou de la liste (dans l'ordre croissant, sauf mention explicite du contraire).
- ❑ Les algorithmes de tris sont parmi les algorithmes les plus utiles, et aussi parmi les exercices d'algorithmiques les plus courants.
- ❑ il existe plusieurs façons différentes connues d'effectuer le tri.

Introduction: Problème de Tri

- ❑ En général, on veut trier des enregistrements avec une clé et des données attachées.



- ❑ Les tableaux permettent de stocker plusieurs éléments de même type au sein d'une seule entité,
- ❑ Lorsque le type de ces éléments possède un ordre total, on peut donc les ranger en ordre croissant ou décroissant,

Introduction: Problème de Tri

- ❑ Trier un tableau c'est donc ranger les éléments d'un tableau en ordre croissant ou décroissant.
- ❑ Dans ce cours on ne fera que des tris en ordre croissant

Le problème de tri :

Entrée : une séquence de n nombres $\langle a_1, a_2, \dots, a_n \rangle$

Sortie : une permutation de la séquence de départ $\langle a'_1, a'_2, \dots, a'_n \rangle$
telle que $a'_1 \leq a'_2 \leq \dots \leq a'_n$

- ❑ Condition: Ensemble totalement ordonné
- ❑ Exemple: relation d'ordre associé à la clé d'une relation en bases de données, ordre sur $\mathbb{N}$, ordre alphabétique,...

Introduction: Problème de Tri

- ❑ C'est un problème simple, connu et extrêmement courant.
- ❑ De nombreuses solutions avec structures de données variées.
- ❑ Sous-problème de problèmes complexes.
- ❑ Il existe plusieurs méthodes de tri qui se différencient par leur complexité d'exécution et leur complexité de compréhension pour le programmeur.

Applications

- ❑ Applications innombrables :
 - ❑ Tri des mails selon leur ancienneté
 - ❑ Tri des résultats de requête dans un moteur de recherche
 - ❑ Tri des facettes des objets pour l'affichage dans les jeux 3D
 - ❑ Gestion des opérations bancaires
 - ❑

Applications

- ❑ Le tri sert aussi de brique de base pour d'autres algorithmes :
 - ❑ Recherche binaire dans un tableau trié
 - ❑ Recherche des éléments dupliqués dans une liste
 - ❑ Recherche du $k^{\text{ème}}$ élément le plus grand dans une liste
 - ❑
- ❑ Des études montrent qu'environ 25% du temps CPU des ordinateurs est utilisé pour trier

Différents types de tri

- ❑ Tri interne : tri en mémoire centrale.
- ❑ Tris externes : données sur un disque externe.
- ❑ Tri de tableau : tri qui trie un tableau. Extensible à toutes structures de données offrant un accès en temps (quasi) constant à ses éléments.
- ❑ Tri générique : peut trier n'importe quel type d'objets pour autant qu'on puisse comparer ces objets.
- ❑ Tri comparatif : basé sur la comparaison entre les éléments (clés)

Différents types de tri

- ❑ Tri itératif : basé sur un ou plusieurs parcours itératifs du tableau
- ❑ Tri récursif : basé sur une procédure récursive
- ❑ Tri en place : modifie directement la structure qu'il est en train de trier. Ne nécessite qu'une quantité très limitée de mémoire supplémentaire.
- ❑ Tri stable : conserve l'ordre relatif des éléments égaux (au sens de la méthode de comparaison).

Tri à bulle: Principe

- ❑ L'idée de départ du tri à bulles consiste à se dire qu'un tableau trié en ordre croissant, c'est un tableau dans lequel tout élément est plus petit que celui qui le suit.
- ❑ Prenons chaque élément d'un tableau, et comparons-le avec l'élément qui le suit:
 - ❑ Si l'ordre n'est pas bon, on permute ces deux éléments et on recommence jusqu'à ce que l'on n'ait plus aucune permutation à effectuer.
 - ❑ Les éléments les plus grands « remontent » ainsi peu à peu vers les dernières places, ce qui explique la dénomination de « tri à bulle ».

Tri à bulle: Principe

- ❑ Mais il ne faut pas oublier un détail capital : à quel moment doit-on s'arrêter?
- ❑ L'idée, c'est que nous déclarons une variable booléenne, cette variable va nous signaler le fait qu'il y a eu au moins une permutation effectuée.
- ❑ Il faut donc :
 - ❑ Lui attribuer la valeur Vrai dès qu'une seule permutation a été faite.
 - ❑ La remettre à Faux à chaque tour de la boucle principale.
 - ❑ Dernier point, il ne faut pas oublier de lancer la boucle principale, et pour cela de donner la valeur Vrai à notre variable booléenne au tout départ de l'algorithme.

Tri à bulle: Algorithme

```
PROCEDURE tri_bulle ( TABLEAU a[1:n])  
passage ← 0  
REPETER  
    permut ← FAUX  
    POUR i VARIANT DE 1 A n - 1 - passage FAIRE  
        SI a[i] > a[i+1] ALORS  
            echanger a[i] ET a[i+1]  
            permut ← VRAI  
        FIN SI  
    FIN POUR  
    passage ← passage + 1  
TANT QUE permut = VRAI
```

Exemple d'exécution

Étape 1

Étape 2

Étape 3

Étape 4

5	1	3	2	4
---	---	---	---	---

1	3	2	4	5
1	2	3	4	5
1	2	3	4	5
1	2	3	4	5

5	1	3	2	4
1	5	3	2	4
1	3	5	2	4
1	3	2	5	4
1	3	2	4	5

Tri à bulle: Complexité

- ❑ Dans le meilleur des cas, avec des données déjà triées, l'algorithme effectuera seulement $n - 1$ comparaisons. Sa complexité dans le meilleur des cas est donc en $O(n)$.
- ❑ Dans le pire des cas, avec des données triées à l'envers, les parcours successifs du tableau imposent d'effectuer:

$$\begin{aligned} C &= (n - 2) + 1 + ([n - 1] - 2) + 1 + \dots + 1 + 0 = \\ &= (n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2} \end{aligned}$$

- ❑ On a donc une complexité dans le pire des cas du tri bulle en $O(n^2)$.

Tri à bulle: implémentation en C

```
void tri_bulle(int* tableau)
{
    int passage = 0;
    bool permutation = true;
    int en_cours;

    while ( permutation ) {
        permutation = false;
        passage ++;
        for (en_cours=0; en_cours<n-1-passage; en_cours++) {
            if (tableau[en_cours]>tableau[en_cours+1]){
                permutation = true;
                // on echange les deux elements
                int temp = tableau[en_cours];
                tableau[en_cours] = tableau[en_cours+1];
                tableau[en_cours+1] = temp;
            }
        }
    }
}
```


Tri par sélection: Principe

tant que il reste plus d'un élément non trié **faire**
chercher le plus petit parmi les non triés ;
échanger le premier élément non trié avec le plus petit trouvé;
Fin;

- Trouver le plus petit élément et le mettre au début de la liste
- Trouver le 2^e plus petit et le mettre en seconde position
- Trouver le 3^e plus petit élément et le mettre à la 3^e place,
-

Tri par sélection: Algorithme

```
PROCEDURE tri_Selection ( Tableau a[1:n])  
  POUR i VARIANT DE 1 A n - 1 FAIRE  
    TROUVER [j] LE PLUS PETIT ELEMENT DE [i + 1:n];  
    ECHANGER [j] ET [i];  
FIN PROCEDURE;
```

Exemple d'exécution

éléments triés				
éléments non triés				
7	8	15	5	10
5	8	15	7	10
5	7	15	8	10
5	7	8	15	10
5	7	8	10	15
5	7	8	10	15

Tri par sélection: Complexité

❑ Cas de comparaisons de deux cellules du tableau

Le nombre de comparaisons "**si $T[j] < T[i]$ alors**" est une valeur qui ne dépend que de n (n est le nombre d'éléments du tableau). la boucle "**pour i de 0 jusqu'à $n-2$ faire**" s'exécute $n-1$ fois, et à chaque itération la boucle "**pour j de $i+1$ jusqu'à $n-1$ faire**" exécute $(n - 1 - (i + 1) + 1)$ fois la comparaison "**si $T[j] < T[i]$ alors**".

La complexité en nombre de comparaison est égale à la somme des $n-1$ termes suivants ($i = 0, \dots, i = n - 2$)

$$\begin{aligned} C &= (n - 2) + 1 + (n - 3) + 1 + \dots + 1 + 0 = \\ &= (n - 1) + (n - 2) + \dots + 1 = \frac{n \cdot (n - 1)}{2} \end{aligned}$$

La complexité en nombre de comparaison est de l'ordre de n^2 , $O(n^2)$.

Tri par sélection: Complexité

❑ Cas de l'échange de deux cellules du tableau

Le cas le plus mauvais est celui où le tableau est déjà classé mais dans l'ordre inverse.

L'échange a lieu systématiquement dans la boucle principale "***pour i de 0 jusqu'à n-2 faire***" qui s'exécute $n-1$ fois : La complexité en nombre d'échanges de cellules est de l'ordre de n , que l'on écrit $O(n)$.

C'est un échange valant 3 affectations la complexité en transfert est $O(3n) = O(n)$

Outre le nombre de comparaison, c'est le nombre d'affectations d'indice qui représente une opération fondamentale et là les deux versions ont exactement la même complexité $O(n^2)$

Tri par sélection: Implémentation en C

```
void tri_selection(int *tableau, int taille)
{
    int en_cours, plus_petit, j, temp;

    for (en_cours = 0; en_cours < taille - 1; en_cours++)
    {
        plus_petit = en_cours;
        for (j = en_cours+1; j < taille; j++) {
            if (tableau[j] < tableau[plus_petit])
                plus_petit = j; }
        temp = tableau[en_cours];
        tableau[en_cours] = tableau[plus_petit];
        tableau[plus_petit] = temp;
    }
}
```

Tri par insertion: Principe

- Ordonner les deux premiers éléments
 - Insérer le 3e élément de manière à ce que les 3 premiers éléments soient triés
 - Insérer le 4e élément à “sa” place pour que. . .
 - . . .
 - Insérer le ne élément à sa place.
-
- À la fin de la i^e itération, les i premiers éléments de T sont triés et rangés au début du tableau T .



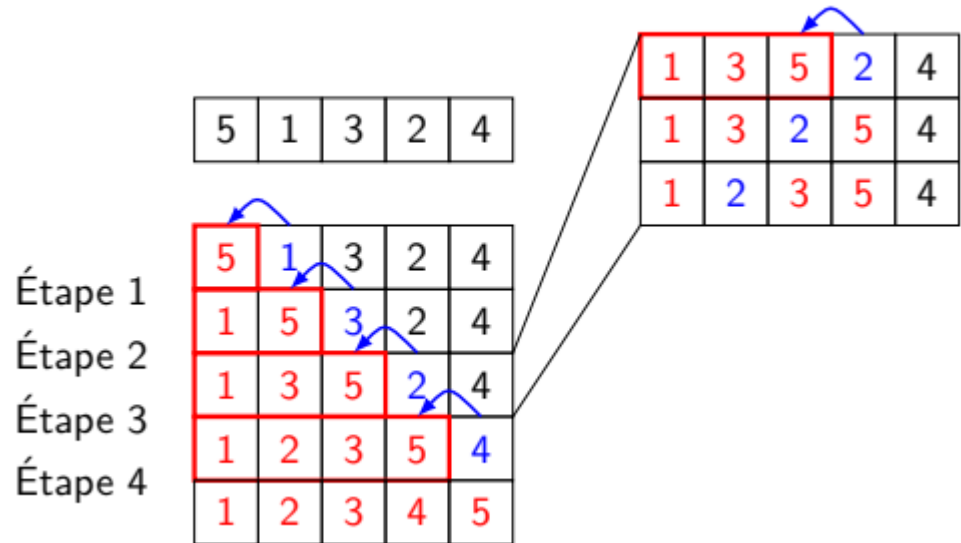
Tri par insertion: Principe

- ❑ Nous insérons, itérativement, le prochain élément dans la partie qui est déjà triée précédemment.
- ❑ La partie de départ qui est triée est le premier élément
- ❑ Les éléments sont placés un à un directement à leur place
- ❑ Attention: En insérant un élément dans la partie triée, il se pourrait qu'on ait à déplacer plusieurs autres.

Tri par insertion: Algorithme

```
PROCEDURE tri_Insertion ( Tableau a[1:n])  
  POUR i VARIANT DE 2 A n FAIRE  
    INSERER a[i] à sa place dans a[1:i-1];  
FIN PROCEDURE;
```

Exemple d'exécution



Tri par insertion: Complexité

- ❑ Comme nous n'avons pas nécessairement à scanner toute la partie déjà triée, le pire cas, le meilleur cas et le cas moyen peuvent différer entre eux
- ❑ **Meilleur cas:** Chaque élément est inséré à la fin de la partie triée. Dans ce cas, nous n'avons à déplacer aucun élément. Comme nous avons à insérer $(n-1)$ éléments, chacun générant seulement une comparaison, la complexité est en $O(n)$.
- ❑ **Pire cas :** Chaque élément est inséré au début de la partie triée. Dans ce cas, tous les éléments de la partie triée doivent être déplacés à chaque itération. La $i^{\text{ème}}$ itération générée $(i - 1)$ comparaisons et échanges de valeurs:

$$\sum_{i=1}^n (i - 1) = \frac{n(n - 1)}{2} = O(n^2)$$

Tri par insertion: Complexité

Remarques:

- ❑ C'est le même nombre de comparaison avec le tri par sélection, mais le tri par insertion effectue plus d'échanges de valeurs. Si les valeurs à échanger sont importantes, ce nombre peut ralentir cet algorithme d'une manière significative.
- ❑ On pourrait essayer d'améliorer cet algorithme en effectuant une recherche dichotomique de la place où insérer l'élément i (dans la première partie triée de la liste), mais la complexité resterait $O(n)$ en raison des déplacements effectués

Tri par insertion: Implémentation en C

```
void tri_insertion(int* t, int taille)
{
    int i, j;
    int en_cours;

    for (i = 1; i < taille ; i++) {
        en_cours = t[i];
        for (j = i; j > 0 && t[j - 1] > en_cours; j--) {
            t[j] = t[j - 1];
        }
        t[j] = en_cours;
    }
}
```

Exercice:

- ❑ Réécrire l'algorithme tri à bulle sans utiliser la boucle répéter ni de variable booléenne

Algorithme : TriBulle(T)

Données : Un tableau T de nombres

Résultat : Le tableau T trié en ordre croissant

```
pour  $i = \text{len}(T) - 1$  à 1 décroissant faire
┌   pour  $j = 0$  à  $i - 1$  faire
┌   ┌   si  $T[j] > T[j + 1]$  alors
┌   ┌   ┌   Echange(T, j, j + 1);
┌   ┌   └
┌   └
└
```

Algorithme : Echange(T, i, j)

Données : Un tableau T de nombres, et deux indices i et j

Résultat : T[i] et T[j] échangés

```
aux ← T[i];
T[i] ← T[j];
T[j] ← aux;
```
